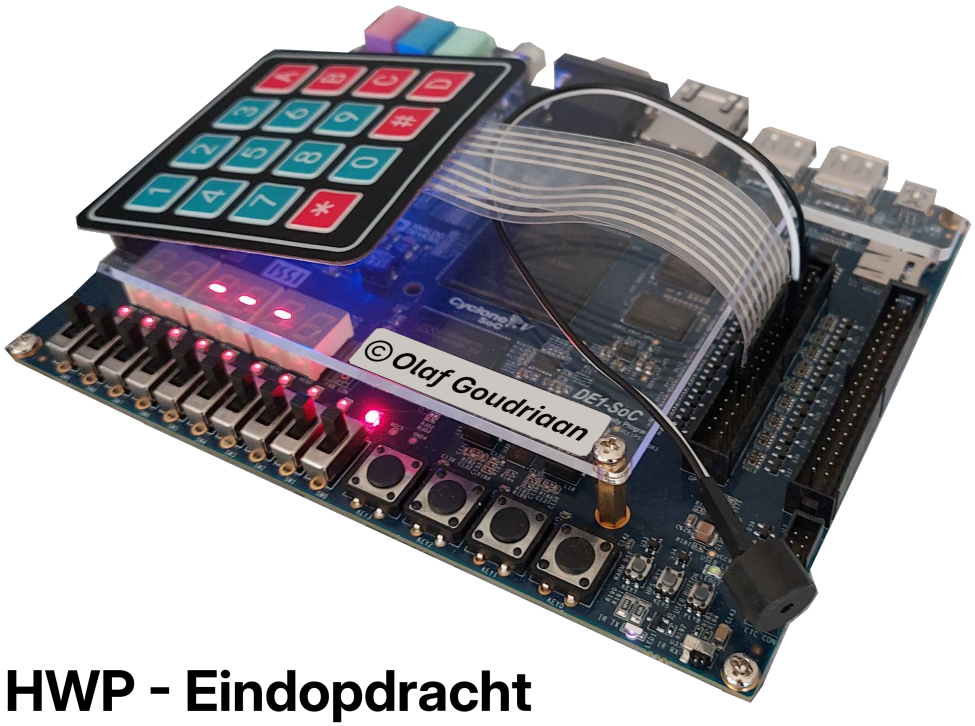


HWP - Eindopdracht

Minor elektrotechniek

Eindopdracht HWP
Embedded Systems

Olaf Goudriaan, 1071349
Fanuel Hadgu, 1087006
2026-2027
Broeders, J.Z.M. (Harry)
ELEHWP10

Inhoudsopgave

1 Werkend prototype **2**

2 Opdracht **3**
2.1 Statemachine . . . 3
2.2 Keypad . . . 3
2.3 Passcode . . . 3
2.4 Buzzer . . . 3
2.5 Sevensegment Driver . . . 3
2.6 Counter . . . 3

3 Ontwerpen **4**

4 VHDL Code **8**
4.1 Keypad . . . 12
4.2 Passcode . . . 16
4.3 Buzzer . . . 19
4.4 Sevensegment driver . . . 22
4.5 Statemachine . . . 23
4.6 Clockdivider . . . 26

1 Werkend prototype

Een youtube link naar het werkende prototype: https://www.youtube.com/shorts/qaX7Bm6lbgI

2 Opdracht

In deze opdracht gaat de HWP opdracht van de keypad uitgebreid worden tot een deurslot met keypad. Ook zal er een buzzer aan toegevoegd worden.

2.1 Statemachine

In het ontwerp zal een statemachine te vinden zijn met vier states. Als het systeem opstart of reset zal deze naar de 'closed' state gaan. Wanneer iemand begint met cijfers invoeren zal de state veranderen naar "progress". Vanuit "progress" kan de state drie kanten op gaan. Met '*' zal de state resetten naar 'closed' en een reset geluidje geven, als de code van 4 cijfers fout is zal de buzzer een fout geluidje geven (signaal is te bepalen), als de code juist is zal er een 'juist signaal' klinken (signaal is te bepalen). Ook zal de led die bij alle andere states aan staat, uit gaan. Na een aantal seconden zal de state terug naar 'closed' gaan. Wanneer de state terug naar closed gaat vanuit fout of correct zal er een geluidje klinken.

Als er een cijfer ingevoerd wordt, zal er een geluidje klinken. En op de sevensegment displayes zal een '-' komen op de plek van het getal.

2.2 Keypad

Voor dit component moet een module gemaakt worden dat controleerd of er een key is ingedrukt en de key doorgeeft als output. Daarnaast geeft deze ook een puls als er een cijfer is ingedrukt. Ook geeft deze de keypad reset signaal, keypad done en pressed signaal.

2.3 Passcode

Dit is een component dat de keys opslaat wanneer deze zijn ingedrukt en daarna controleerd of de cijfercode correct is.

2.4 Buzzer

Dit is een component dat de buzzer aanstuurt om de geluiden te geven. Je kunt selecteren welk geluid door het geluid te selecteren. De geluiden zijn: 'key pressed', 'code fault', 'code correct' en 'reset'. De exacte geluiden voor deze signalen moeten nog bepaald worden.

2.5 Sevensegment Driver

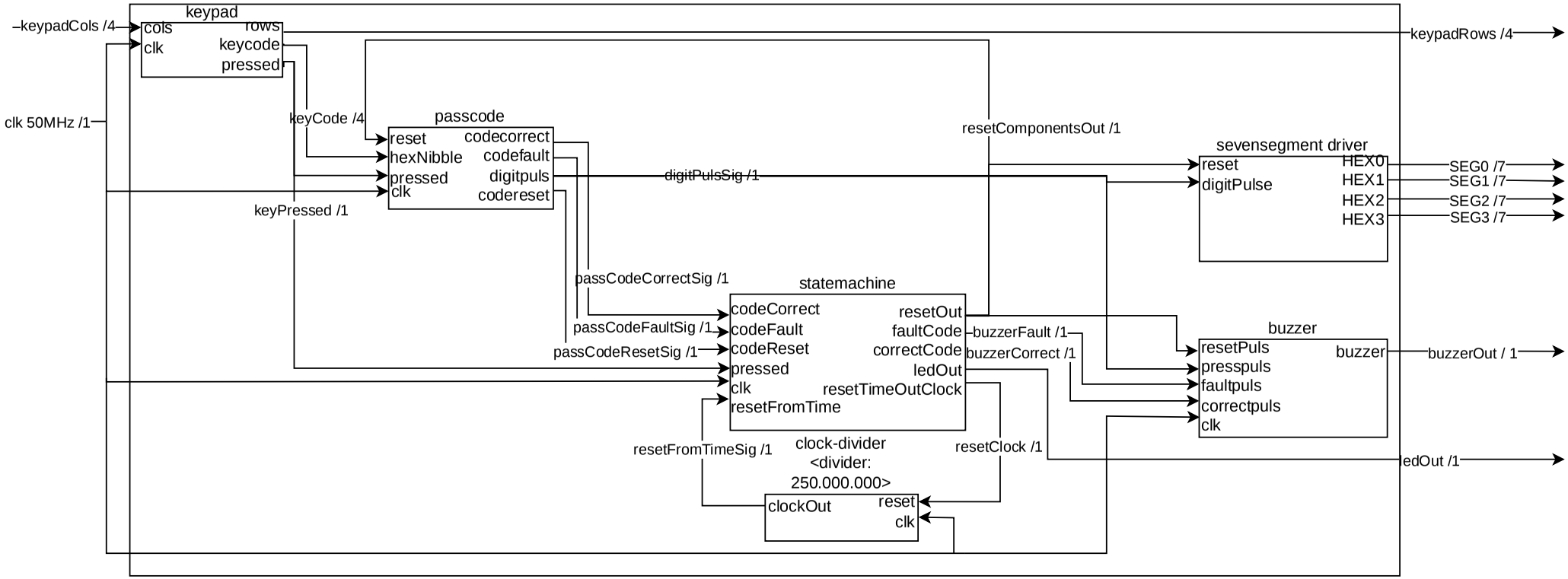
Deze driver moet de sevensegments aansturen. En bij iedere cijferspuls zal deze een extra '-' op een display zetten.

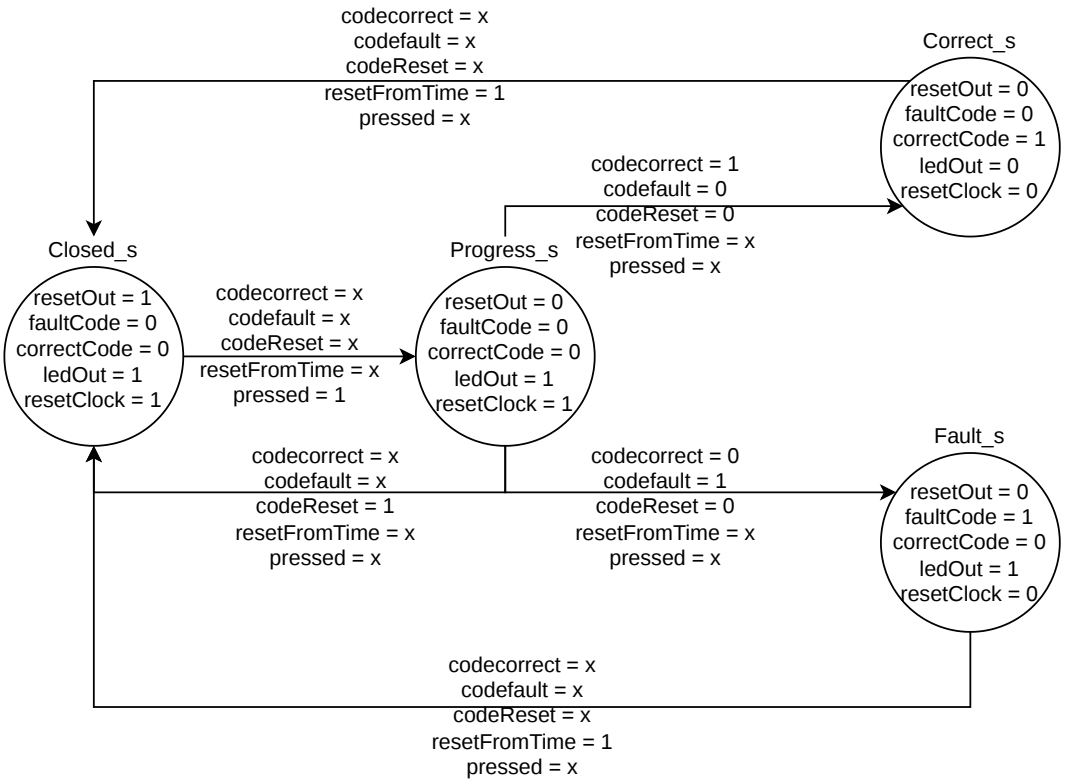
2.6 Counter

Om terug te gaan naar de closed state wordt een counter gebruikt.

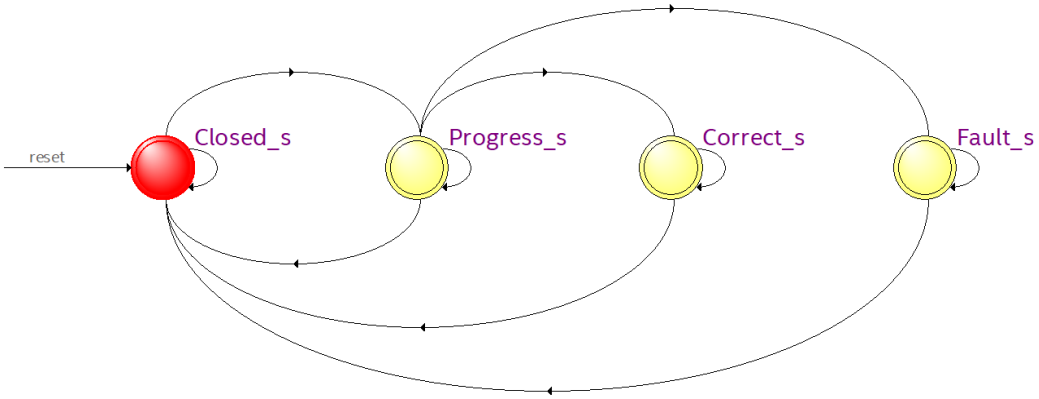
3 Ontwerpen

Hieronder zijn de hoofd diagrammen te zien van het systeem. Dit zijn het toetstandsdiagram en het blokdiagram.


keypad
cols
clk
rows
keycode
pressed
keypadCols /4
keypadRows /4
clk 50MHz /1
keyCode /4
keyPressed /1
passcode
reset
hexNibble
pressed
clk
codecorrect
codefault
digitpuls
codereset
resetComponentsOut /1
digitPulsSig /1
sevensegment driver
reset
digitPulse
HEX0
HEX1
HEX2
HEX3
SEG0 /7
SEG1 /7
SEG2 /7
SEG3 /7
passCodeCorrectSig /1
passCodeFaultSig /1
passCodeResetSig /1
statemachine
codeCorrect
codeFault
codeReset
pressed
clk
resetFromTime
resetOut
faultCode
correctCode
ledOut
resetTimeOutClock
buzzerFault /1
buzzerCorrect /1
buzzer
resetPuls
presspuls
faultpuls
correctpuls
clk
buzzer
buzzerOut / 1
ledOut /1
resetFromTimeSig /1
clock-divider
<divider:
250.000.000>
resetClock /1
clockOut
reset
clk


codecorrect = x
codefault = x
codeReset = x
resetFromTime = 1
pressed = x
Correct_s
resetOut = 0
faultCode = 0
correctCode = 1
ledOut = 0
resetClock = 0
codecorrect = 1
codefault = 0
codeReset = 0
resetFromTime = x
pressed = x
Closed_s
resetOut = 1
faultCode = 0
correctCode = 0
ledOut = 1
resetClock = 1
codecorrect = x
codefault = x
codeReset = x
resetFromTime = x
pressed = 1
Progress_s
resetOut = 0
faultCode = 0
correctCode = 0
ledOut = 1
resetClock = 1
Fault_s
codecorrect = x
codefault = x
codeReset = 1
resetFromTime = x
pressed = x
codecorrect = 0
codefault = 1
codeReset = 0
resetFromTime = x
pressed = x
resetOut = 0
faultCode = 1
correctCode = 0
ledOut = 1
resetClock = 0
codecorrect = x
codefault = x
codeReset = x
resetFromTime = 1
pressed = x

De statemachine is vergeleken met de statemachine vanuit quartus software. Deze is identiek aan de statemachine die vooraf was ontworpen. Deze is hieronder te zien.



Daarnaast is voor de buzzer ook nog een toestandsdiagram gemaakt, Deze is hieronder te vinden.

4 VHDL Code

De volledige code is hieronder te zien.

Dit stukje code is concurrent omdat dit alleen maar de verschillende delen met elkaar verbind

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity keypadtoplevel is
  port(
    keypadCols: in std_ulogic_vector(3 downto 0);
    clk: in std_ulogic;

    SEG0: out std_ulogic_vector(6 downto 0);
    SEG1: out std_ulogic_vector(6 downto 0);
    SEG2: out std_ulogic_vector(6 downto 0);
    SEG3: out std_ulogic_vector(6 downto 0);
    keypadRows: out std_ulogic_vector(3 downto 0);
    ledOut: out std_ulogic;
    buzzerOut: out std_ulogic
  );
end entity;

architecture behavioral of keypadtoplevel is
  component ClockDivider is
    Generic (
      counter : natural := 250000000
    );
    Port (
      reset: in std_ulogic;
      clockOut: out std_ulogic;
      clk : in std_ulogic
    );
  end component;

  component keypad is
    port(
      cols: in std_ulogic_vector(3 downto 0);
      rows: out std_ulogic_vector(3 downto 0);
      clk: in std_ulogic;

      pressed: out std_ulogic;
      keycode: out std_ulogic_vector(3 downto 0)
    );
  end component;

  component passcode is
    port (
      clk: in std_ulogic;
      reset: in std_ulogic;
      hexNibble: in std_ulogic_vector(3 downto 0);
      pressed: in std_ulogic;

```

```
49        codeCorrect: out std_ulogic;
50        codeFault: out std_ulogic;
51        digitPuls: out std_ulogic;
52        codeReset: out std_ulogic
53      );
54    end component;
55
56    component seven_segment_driver is
57      port (
58        reset: in  std_ulogic;
59        digitPuls: in  std_ulogic;
60        HEX0: out std_ulogic_vector(6 downto 0);
61        HEX1: out std_ulogic_vector(6 downto 0);
62        HEX2: out std_ulogic_vector(6 downto 0);
63        HEX3: out std_ulogic_vector(6 downto 0)
64      );
65    end component;
66
67    component statemachine is
68      port (
69        codeCorrect: in std_ulogic;
70        codeFault: in std_ulogic;
71        codeReset: in std_ulogic;
72        resetFromTime: in std_ulogic;
73        clk: in std_ulogic;
74        pressed: in std_ulogic;
75
76        resetOut: out std_ulogic;
77        faultCode: out std_ulogic;
78        correctCode: out std_ulogic;
79        ledOut: out std_ulogic;
80        resetTimeOutClock: out std_ulogic
81      );
82    end component;
83
84    component buzzer is
85      port(
86        clk: in  std_ulogic;
87        resetPuls: in  std_ulogic;
88        pressPuls: in  std_ulogic;
89        faultPuls: in  std_ulogic;
90        correctPuls: in  std_ulogic;
91        buzzer: out std_ulogic
92      );
93    end component;
94
95    -- signals
96    signal keyPressed : std_ulogic;
97    signal keyCode : std_ulogic_vector(3 downto 0);
98    signal passCodeCorrectSig : std_ulogic;
99    signal passCodeFaultSig : std_ulogic;
100   signal passCodeResetSig : std_ulogic;
101   signal digitPulsSig : std_ulogic;
102   signal resetFromTimeSig : std_ulogic;
```

```
103
104   -- output statemachine
105   signal resetComponentsOut : std_ulogic;
106   signal buzzerFault : std_ulogic;
107   signal buzzerCorrect : std_ulogic;
108   signal resetClock : std_ulogic;
109 begin
110
111   clockdividermodule: ClockDivider generic map (
112     counter => 250000000
113   ) port map (
114     reset => resetClock,
115     clockOut => resetFromTimeSig,
116     clk => clk
117   );
118
119   statemachinemodule : statemachine port map (
120     codeCorrect => passCodeCorrectSig,
121     codeFault => passCodeFaultSig,
122     codeReset => passCodeResetSig,
123     pressed => keyPressed,
124     clk => clk,
125     resetFromTime => resetFromTimeSig,
126
127     resetOut => resetComponentsOut,
128     faultCode => buzzerFault,
129     correctCode => buzzerCorrect,
130     ledOut => ledOut,
131     resetTimeOutClock => resetClock
132   );
133
134   passcodemodule : passcode port map (
135     clk => clk,
136     reset => resetComponentsOut,
137     hexNibble => keyCode,
138     pressed => keyPressed,
139
140     codeCorrect => passCodeCorrectSig,
141     codeFault => passCodeFaultSig,
142     digitPuls => digitPulsSig,
143     codeReset => passCodeResetSig
144   );
145
146   sevensegmodule : seven_segment_driver port map (
147     reset => resetComponentsOut,
148     digitPuls => digitPulsSig,
149     HEX0 => SEG0,
150     HEX1 => SEG1,
151     HEX2 => SEG2,
152     HEX3 => SEG3
153   );
154
155   keypadmodule : keypad port map (
156     cols => keypadCols,
```

```
157     rows => keypadRows,
158     clk => clk,
159     pressed => keyPressed,
160     keycode => keyCode
161   );
162 
163   buzzermodule : buzzer port map (
164     clk => clk,
165     resetPuls => resetComponentsOut,
166     pressPuls => digitPulsSig,
167     faultPuls => buzzerFault,
168     correctPuls => buzzerCorrect,
169     buzzer => buzzerOut
170   );
171 
172 end architecture;
```

Listing 1: keypadtoplevel.vhd

4.1 Keypad

Het volgende stuk code is voor de keypad, er is besloten dat deze alleen een puls stuurt wanneer er een key is ingedrukt en daarnaast altijd de keycode, dit is omdat alle andere componenten dan geen rekening hoeven te houden met hoe de keypad werkt. Deze code is sequential omdat deze op een frequentie van 100hz per rij werkt omdat dat meer dan genoeg is.

```
library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;

entity keypad is
  port(
    cols     : in  std_ulogic_vector(3 downto 0);
    rows     : out std_ulogic_vector(3 downto 0);
    clk      : in  std_ulogic;

    pressed  : out std_ulogic;
    keycode  : out std_ulogic_vector(3 downto 0)
  );
end entity;

architecture behavioral of keypad is

  component ClockDivider is
    generic (
      counter : natural := 25000000
    );
    port (
      reset    : in  std_ulogic;
      clockOut : out std_ulogic;
      clk      : in  std_ulogic
    );
  end component;

  signal currentRow : integer range 0 to 3 := 0;

  signal scanRow  : std_ulogic;
  signal reset    : std_ulogic := '0';
  signal outClock : std_ulogic;

  signal lastKey    : std_ulogic_vector(3 downto 0) := (others => '0');
  signal currentKey : std_ulogic_vector(3 downto 0) := (others => '0');

  signal detected   : std_ulogic := '0';
  signal keyReleased : std_ulogic := '1';

begin

  clockInst : ClockDivider
    generic map (
      counter => 125000
    )
    port map (
      reset    => reset,
```

```
48        clockOut => scanRow,
49        clk      => clk
50      );
51
52    clockInst2 : ClockDivider
53      generic map (
54        counter => 5000
55      )
56      port map (
57        reset    => reset,
58        clockOut => outClock,
59        clk      => clk
60      );
61
62    ------------------------------------------------
63    -- Keypad scanning process
64    ------------------------------------------------
65
66    process(scanRow)
67
68      variable vKey      : std_ulogic_vector(3 downto 0);
69      variable vDetected : std_ulogic;
70
71    begin
72
73      if rising_edge(scanRow) then
74        vKey := currentKey;
75        vDetected := detected;
76
77        if currentRow = 0 then
78          vDetected := '0';
79          vKey := "1111";
80        end if;
81
82        case currentRow is
83          when 0 => rows <= "1110";
84          when 1 => rows <= "1101";
85          when 2 => rows <= "1011";
86          when 3 => rows <= "0111";
87          when others => rows <= "1111";
88        end case;
89
90        case cols is
91          when "0111" =>
92            case currentRow is
93              when 0 =>
94                vKey := "0001";
95                vDetected := '1';
96
97              when 1 =>
98                vKey := "1110";
99                vDetected := '1';
100
101             when 2 =>
```

```
102               vKey := "0111";
103               vDetected := '1';
104
105             when 3 =>
106               vKey := "0100";
107               vDetected := '1';
108
109             when others => null;
110           end case;
111
112         when "1011" =>
113           case currentRow is
114             when 0 =>
115               vKey := "0010";
116               vDetected := '1';
117
118             when 1 =>
119               vKey := "0000";
120               vDetected := '1';
121
122             when 2 =>
123               vKey := "1000";
124               vDetected := '1';
125
126             when 3 =>
127               vKey := "0101";
128               vDetected := '1';
129
130             when others => null;
131           end case;
132
133         when "1101" =>
134           case currentRow is
135             when 0 =>
136               vKey := "0011";
137               vDetected := '1';
138
139             when 2 =>
140               vKey := "1001";
141               vDetected := '1';
142
143             when 3 =>
144               vKey := "0110";
145               vDetected := '1';
146
147             when others => null;
148           end case;
149         when others => null;
150       end case;
151
152       currentKey <= vKey;
153       detected <= vDetected;
154
155       if currentRow = 3 then
```

```
156         currentRow <= 0;
157         if vDetected = '1' then
158           if keyReleased = '1' or lastKey /= vKey then
159             pressed <= '1';
160             lastKey <= vKey;
161             keycode <= vKey;
162             keyReleased <= '0';
163           else
164             pressed <= '0';
165           end if;
166         else
167           if cols = "1111" then
168             keyReleased <= '1';
169           end if;
170           pressed <= '0';
171         end if;
172       else
173         currentRow <= currentRow + 1;
174       end if;
175     end if;
176   end process;
177
178 end architecture;
```

Listing 2: keypad.vhd

4.2 Passcode

De passcode reageert alleen maar op pulses van de de module van de keypad, Deze module wordt gereset in de idle state van de statemachine. De code is opgeslagen in een 16 bit getal, 4 bits per cijfer. De inputs worden aan elkaar vastgeknoopt door and operatoren, waardoor de code 16 bits wordt. Deze wordt dan gecheckt met de juiste code. Als de code juist is zal het signaal voor code juist worden gestuurt, is die fout zal het signaal voor code fout gestuurt worden. Als deze module '*' (1110) krijgt, zal die de reset output geven. Dit reset het systeem waardoor de code gereset wordt. Deze module stuurt ook signalen door naar de sevensegment driver en de buzzer om te laten weten dat er een valid key is ingedrukt. Deze code is sequential omdat deze werkt met een klok.

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity passcode is
  port (
    clk: in std_ulogic;
    reset: in std_ulogic;
    hexNibble: in std_ulogic_vector(3 downto 0);
    pressed: in std_ulogic;

    codeCorrect: out std_ulogic;
    codeFault: out std_ulogic;
    digitPuls: out std_ulogic;
    codeReset: out std_ulogic
  );
end entity;

architecture behavioral of passcode is
  signal count        : integer range 0 to 4 := 0;

  signal inputdig0    : std_ulogic_vector(3 downto 0) := "0000";
  signal inputdig1    : std_ulogic_vector(3 downto 0) := "0000";
  signal inputdig2    : std_ulogic_vector(3 downto 0) := "0000";

  constant correctCode : std_ulogic_vector(15 downto 0) :=
   "0001001010010011";

  -- current and last signal from "pressed"
  signal pressedLast   : std_ulogic := '0';
  signal pressedCurrent : std_ulogic := '0';
begin

  -- rising and falling edge detection for 'pressed' signal
  process(clk, reset)
  begin
    if reset = '1' then
      pressedLast   <= '0';
      pressedCurrent <= '0';
    elsif rising_edge(clk) then
      pressedLast   <= pressed;
      pressedCurrent <= pressed and not pressedLast; -- Single-cycle pulse
    on button press
    end if;
  end process;
```

```
43
44    process(clk, reset)
45    begin
46      if reset = '1' then
47        inputdig0   <= "0000";
48        inputdig1   <= "0000";
49        inputdig2   <= "0000";
50        codeReset   <= '0';
51        codeCorrect <= '0';
52        codeFault   <= '0';
53        digitPuls   <= '0';
54        count       <= 0;
55      elsif rising_edge(clk) then
56        codeReset   <= '0';
57        codeCorrect <= '0';
58        codeFault   <= '0';
59        digitPuls   <= '0';
60
61        if pressedCurrent = '1' then
62          if hexNibble = "1110" then
63            codeReset <= '1';
64            count     <= 0;
65          else
66            digitPuls <= '1';
67            case count is
68              when 0 =>
69                inputdig0 <= hexNibble;
70                count     <= 1;
71              when 1 =>
72                inputdig1 <= hexNibble;
73                count     <= 2;
74              when 2 =>
75                inputdig2 <= hexNibble;
76                count     <= 3;
77              when 3 =>
78                count     <= 4;
79                if (inputdig0 & inputdig1 & inputdig2 & hexNibble) =
    correctCode then
80                  codeCorrect <= '1';
81                else
82                  codeFault <= '1';
83                end if;
84
85              when others =>
86                codeFault <= '1';
87                count     <= 0;
88            end case;
89          end if;
90        end if;
91      end if;
92    end process;
93
94  end architecture;
```

Listing 3: passcode.vhd

4.3 Buzzer

De buzzer is een module voor de externe buzzer die aangesloten is op het ontwerp. Deze zit op de GPIO_0_D0 pin. De buzzer heeft een statemachine, die te zien is in het ontwerp hoofdstuk. Deze code zorgt voor dat er piepjes komen als er een code wordt intoetst. Deze module heeft een klok input, en is sequentieel, en heeft vier ander inputs: reset, press, fault en correct. Deze spelen hun geluidjes af wanneer de pin hoog wordt. De press key bij iedere toets, de reset key als het systeem gereset wordt, de correct key als de code juist is en de fault key als de code fout is.

```
0   /* Buzzer-component: Een buzzer gaat aan wanneer er op de keypad wordt
      gedrukt, */
1  /*  wanneer de code goed/fout is, en waneer er op de reset-knop wordt
      gedrukt.  */
2 /*   Hiervoor zijn drie verschillende vormen van geluid bedacht.
          */
3 library ieee;
4 use ieee.std_logic_1164.all;
5
6 entity buzzer is
7     generic (
8         clkFreq: natural := 50000000
9     );
10    port (
11        clk: in std_ulogic;
12        pressPuls: in std_ulogic;
13        faultPuls: in std_ulogic;
14        correctPuls: in std_ulogic;
15        resetPuls: in std_ulogic;
16        buzzer: out std_ulogic -- Geluid is te horen wanneer 'buzzer' hoog
      is.
17    );
18 end entity;
19
20 architecture behavorial of buzzer is
21    constant SHORTBUZZ: natural := clkFreq / 8; -- Korte piep: buzzer gaat
      aan voor 125 milliseconden.
22    constant BUZZGAP: natural := (clkFreq / 8) * 3; -- Tussen twee korte
     piepjes is er een pauze van 375 milliseconden.
23    constant LONGBUZZ: natural := clkFreq / 2;-- Lange piep: buzzer gaat
     aan voor 500 milliseconden.
24
25    type state_type is (idle, buzz1, gap, buzz2);
26    signal state : state_type := idle;
27    signal doubleBuzz: std_ulogic := '0';
28    signal timer: natural range 0 to BUZZGAP + LONGBUZZ := 0;
29
30    signal s1, s2, prev: std_ulogic_vector(3 downto 0) := (others => '0');
31 begin
32    process (clk)
33        variable rise: std_ulogic_vector(3 downto 0);
34    begin
35        if rising_edge(clk) then
36            s1 <= correctPuls & faultPuls & resetPuls & pressPuls;
37            s2 <= s1;
38            prev <= s2;
```

```
39              rise := s2 and not prev;
40
41          if rise(3) = '1' then -- code correct: 2 korte piepjes
42              state <= buzz1;
43              timer <= SHORTBUZZ - 1;
44              doubleBuzz <= '1';
45          elsif rise(2) = '1' then -- code fout: 1 lange piep
46              state <= buzz1;
47              timer <= LONGBUZZ - 1;
48              doubleBuzz <= '0';
49          elsif rise(1) = '1' then -- reset code: 1 lange piep
50              state <= buzz1;
51              timer <= LONGBUZZ - 1;
52              doubleBuzz <= '0';
53          elsif rise(0) = '1' then -- knop ingedrukt: 1 korte piep
54              state <= buzz1;
55              timer <= SHORTBUZZ - 1;
56              doubleBuzz <= '0';
57          else
58              case state is
59                  when idle =>
60                      null;
61                  when buzz1 =>
62                      if timer = 0 then
63                          if doubleBuzz = '1' then
64                              state <= gap;
65                              timer <= BUZZGAP - 1;
66                          else
67                              state <= idle;
68                          end if;
69                      else
70                          timer <= timer - 1;
71                      end if;
72                  when gap =>
73                      if timer = 0 then
74                          state <= buzz2;
75                          timer <= SHORTBUZZ - 1;
76                      else
77                          timer <= timer - 1;
78                      end if;
79                  when buzz2 =>
80                      if timer = 0 then
81                          state <= idle;
82                      else
83                          timer <= timer - 1;
84                      end if;
85                  end case;
86              end if;
87          end if;
88      end process;
89
90      buzzer <= '1' when (state = buzz1 or state = buzz2) else '0';
91  end architecture;
```

Listing 4: buzzer.vhd

4.4 Sevensegment driver

Deze module zet streepjes op het scherm wanneer deze een puls krijgt van de passcode module. Deze wordt gereset door de statemachine door middel van de rest pin. Deze module telt het aantal pulsen dat die krijgt en laat een dash zien op de segment display die er bij hoort. Deze module telt het aantal pulsen en bij vier gaat die niet meer hoger en wordt pas nul als die gerest wordt. Deze module is ook concurrent omdat deze niet geklokt wordt, hij hoogt alleen een getal op, en als het getal hoog genoeg is laat hij een dash zien.

```
0  library ieee;
1  use ieee.std_logic_1164.all;
2
3  entity seven_segment_driver is
4      port (
5          reset: in  std_ulogic;
6          digitPuls: in  std_ulogic;
7          HEX0: out std_ulogic_vector(6 downto 0);
8          HEX1: out std_ulogic_vector(6 downto 0);
9          HEX2: out std_ulogic_vector(6 downto 0);
10         HEX3: out std_ulogic_vector(6 downto 0)
11     );
12 end entity;
13
14 architecture behavioral of seven_segment_driver is
15     constant DASH: std_ulogic_vector(6 downto 0) := "0111111";
16     constant BLANK: std_ulogic_vector(6 downto 0) := "1111111";
17
18     signal count: natural range 0 to 4 := 0;
19 begin
20     process (digitPuls, reset)
21     begin
22         if reset = '1' then
23             count <= 0;
24         elsif rising_edge(digitPuls) then
25             if count < 4 then
26                 count <= count + 1;
27             end if;
28         end if;
29     end process;
30
31     HEX0 <= DASH when count >= 1 else BLANK;
32     HEX1 <= DASH when count >= 2 else BLANK;
33     HEX2 <= DASH when count >= 3 else BLANK;
34     HEX3 <= DASH when count >= 4 else BLANK;
35 end architecture;
```

Listing 5: seven_segment_driver

4.5 Statemachine

De statemachine heeft 4 states, Closed, Progress, Correct en fault. Hoe de states werken is te zien in het statediagram. De statemachine is sequentieel.

```
library ieee;
use IEEE.numeric_std.all;
use IEEE.std_logic_1164.all;

entity statemachine is
  port (
    codeCorrect: in std_ulogic;
    codeFault: in std_ulogic;
    codeReset: in std_ulogic;
    resetFromTime: in std_ulogic;
    clk: in std_ulogic;
    pressed: in std_ulogic;

    resetOut: out std_ulogic;
    faultCode: out std_ulogic;
    correctCode: out std_ulogic;
    ledOut: out std_ulogic;
    resetTimeOutClock: out std_ulogic
  );
end entity;

architecture structural of statemachine is
  type stateType is (Closed_s, Progress_s, Correct_s, Fault_s);
  signal currentState, nextState: stateType;
begin

  flipflops : process (clk)
  begin
    if rising_edge(clk) then
      currentState <= nextState;
    end if;
  end process;

  process(clk, codeCorrect, codeFault, codeReset, pressed, resetFromTime)
  begin
    case currentState is
      when Closed_s =>
        if pressed = '1' then
          nextState <= Progress_s;
        else
          nextState <= Closed_s;
        end if;

      when Progress_s =>
        if codeCorrect = '1' and not (codeFault = '1' or codeReset = '1')
   then
          nextState <= Correct_s;
        elsif codeFault = '1' and not (codeCorrect = '1' or codeReset =
   '1') then
          nextState <= Fault_s;
```

```
48          elsif codeReset = '1' then
49            nextState <= Closed_s;
50          else
51            nextState <= Progress_s;
52          end if;
53
54        when Correct_s =>
55          if resetFromTime = '1' then
56            nextState <= Closed_s;
57          else
58            nextState <= Correct_s;
59          end if;
60
61        when Fault_s =>
62          if resetFromTime = '1' then
63            nextState <= Closed_s;
64          else
65            nextState <= Fault_s;
66          end if;
67      end case;
68    end process;
69
70    process(currentState)
71    begin
72      case currentState is
73        when Closed_s =>
74          resetOut <= '1';
75          faultCode <= '0';
76          correctCode <= '0';
77          ledOut <= '1';
78          resetTimeOutClock <= '1';
79
80        when Progress_s =>
81          resetOut <= '0';
82          faultCode <= '0';
83          correctCode <= '0';
84          ledOut <= '1';
85          resetTimeOutClock <= '1';
86
87          when Correct_s =>
88          resetOut <= '0';
89          faultCode <= '0';
90          correctCode <= '1';
91          ledOut <= '0';
92          resetTimeOutClock <= '0';
93
94          when Fault_s =>
95          resetOut <= '0';
96          faultCode <= '1';
97          correctCode <= '0';
98          ledOut <= '1';
99          resetTimeOutClock <= '0';
100
101     end case;
```

```
102   end process;
103 end architecture structural;
```

Listing 6: statemachine.vhd

4.6 Clockdivider

De clockdivider die bij deze opdracht is gebruikt is dezelfde als die gebruikt is bij de weekelijkse opgaven.

```
library ieee;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity ClockDivider is
  generic (
    counter: natural := 25000000
  );
  port (
    reset: in std_ulogic;
    clockOut: out std_ulogic;
    clk : in std_ulogic
  );
end entity;

architecture behavoural of ClockDivider is

  signal clockSig : std_ulogic := '0';
  signal count : integer := 1;

begin
  process(clk, reset)
  begin
    if(reset = '1') then
      count <= 1;
      clockSig <= '0';
    elsif(rising_edge(clk)) then
      count <= count + 1;
      if (count = counter) then
        clockSig <= not clockSig;
        count <= 1;
      end if;
    end if;
    clockOut <= clockSig;
  end process;
end architecture;
```

Listing 7: ClockDivider.vhd